PADRÕES DE PROJETO EM JAVA

Guia detalhado: Factory Method, Abstract Factory, Builder, Prototype e Singleton

Conceitos, estrutura, exemplos de código, vantagens, limitações e comparação entre os cinco padrões.

Material de estudo com exemplos em Java moderno, priorizando clareza e aplicação prática.

Como usar este guia

Este material apresenta cinco padrões de criação (creational design patterns), conhecidos por organizar a forma como objetos são criados. Os exemplos usam Java e podem ser adaptados para projetos próprios. Os trechos são independentes: cada exemplo deve ser colocado em arquivos compatíveis com os nomes das classes públicas.

O que são padrões de projeto?

Padrões de projeto são soluções recorrentes para problemas comuns de organização de software. Não são bibliotecas prontas nem regras que precisam ser aplicadas sempre. Eles oferecem uma estrutura de colaboração entre classes que pode tornar o código mais flexível, compreensível e fácil de testar.

Os cinco padrões em uma frase

- Factory Method: deixa subclasses ou implementações concretas decidirem qual objeto será criado por um método de fábrica.
- Abstract Factory: cria famílias de objetos relacionados sem expor suas classes concretas ao cliente.
- Builder: constrói objetos complexos em etapas, separando a construção da representação final.
- Prototype: cria novos objetos copiando um objeto existente, chamado protótipo.
- Singleton: garante uma única instância acessível de uma classe dentro de determinado contexto de execução.

> Atenção
> Um padrão não é automaticamente melhor do que uma solução simples. Se uma classe ou um construtor resolve o problema com clareza, adicionar abstrações pode aumentar a complexidade sem benefício.

Antes dos exemplos

Para manter o foco no padrão, os exemplos não incluem todos os detalhes de uma aplicação real, como persistência em banco, validação completa, logs ou tratamento de erros de infraestrutura. Os códigos ilustram a estrutura e a intenção do padrão.

1. Factory Method

Problema que resolve

Imagine um sistema de notificações que pode enviar mensagens por e-mail ou SMS. Se o código principal instancia diretamente `EmailNotification`, fica acoplado a essa implementação. Para adicionar outro canal, pode ser necessário alterar o código que coordena o envio.

Ideia central

O Factory Method define um método responsável por criar um objeto, mas permite que implementações concretas escolham qual classe instanciar. Em sua forma clássica, existe um criador abstrato ou base com um método de fábrica, e subclasses sobrescrevem esse método para fornecer produtos diferentes.

Estrutura

- Product: interface comum dos objetos criados.
- Concrete Product: implementação concreta do produto.
- Creator: declara o Factory Method e pode conter lógica que usa o produto.
- Concrete Creator: implementa o Factory Method e escolhe o produto concreto.

Exemplo em Java

```
interface Notificacao {
    void enviar(String mensagem);
}

class NotificacaoEmail implements Notificacao {
    @Override
    public void enviar(String mensagem) {
        System.out.println("E-mail enviado: " + mensagem);
    }
}

class NotificacaoSms implements Notificacao {
    @Override
    public void enviar(String mensagem) {
        System.out.println("SMS enviado: " + mensagem);
    }
}

abstract class ServicoNotificacao {
    // Factory Method: a subclasse decide o produto.
    protected abstract Notificacao criarNotificacao();

    public void notificar(String mensagem) {
        Notificacao notificacao = criarNotificacao();
        notificacao.enviar(mensagem);
    }
}

class ServicoEmail extends ServicoNotificacao {
    @Override
    protected Notificacao criarNotificacao() {
        return new NotificacaoEmail();
    }
}

class ServicoSms extends ServicoNotificacao {
    @Override
    protected Notificacao criarNotificacao() {
        return new NotificacaoSms();
    }
}

public class Main {
    public static void main(String[] args) {
        ServicoNotificacao servico = new ServicoEmail();
        servico.notificar("Sua inscrição foi confirmada.");

        servico = new ServicoSms();
        servico.notificar("Seu código é 1234.");
    }
}
```

Como o fluxo funciona

- O cliente escolhe um criador concreto, como `ServicoEmail`.
- O método `notificar()` chama `criarNotificacao()` sem conhecer a classe concreta.
- A subclasse devolve uma implementação de `Notificacao`.
- O serviço usa a interface comum para enviar a mensagem.

Vantagens

- Reduz o acoplamento entre o código cliente e as classes concretas criadas.
- Facilita adicionar novos tipos de produto sem modificar a lógica comum de uso.
- Centraliza a decisão de criação em um ponto especializado.

Limitações

- Pode aumentar o número de classes, pois cada variação pode exigir um criador concreto.

- Se há poucas variações e nenhuma expectativa de crescimento, um construtor simples pode ser suficiente.
- O padrão clássico depende de herança; em alguns sistemas, composição e injeção de dependências são alternativas mais simples.

Quando usar

Use quando uma classe precisa trabalhar com uma interface de produto, mas a escolha da implementação deve variar conforme o tipo de criador. Exemplos: exportadores de arquivos, conectores de pagamento, notificações e integrações com serviços externos.

> Não confunda
> Factory Method não é simplesmente qualquer método chamado `factory`. O padrão clássico usa um método de criação que pode ser redefinido por subclasses.

2. Abstract Factory

Problema que resolve

Um sistema pode oferecer temas visuais diferentes, como claro e escuro. Cada tema possui um conjunto de componentes que combinam entre si: botão e caixa de diálogo. Criar componentes de temas diferentes por engano pode produzir uma interface inconsistente.

Ideia central

A Abstract Factory fornece uma interface para criar uma família de objetos relacionados, sem exigir que o cliente conheça suas classes concretas. Ao escolher uma fábrica concreta, o cliente recebe produtos compatíveis entre si.

Estrutura

- Abstract Factory: declara métodos para criar cada tipo de produto da família.
- Concrete Factory: cria os produtos concretos de uma variante.
- Abstract Products: interfaces dos tipos de objeto da família.
- Concrete Products: implementações específicas de cada variante.
- Client: depende das interfaces, e não das classes concretas.

Exemplo em Java: componentes de interface

```
interface Botao {
    void renderizar();
}

interface CaixaDialogo {
    void abrir();
}

class BotaoClaro implements Botao {
    public void renderizar() {
        System.out.println("Botão com tema claro");
    }
}

class DialogoClaro implements CaixaDialogo {
    public void abrir() {
        System.out.println("Diálogo com tema claro");
    }
}

class BotaoEscuro implements Botao {
    public void renderizar() {
        System.out.println("Botão com tema escuro");
    }
}

class DialogoEscuro implements CaixaDialogo {
    public void abrir() {
        System.out.println("Diálogo com tema escuro");
    }
}

interface FabricaUI {
    Botao criarBotao();
    CaixaDialogo criarDialogo();
}

class FabricaTemaClaro implements FabricaUI {
    public Botao criarBotao() {
        return new BotaoClaro();
    }

    public CaixaDialogo criarDialogo() {
        return new DialogoClaro();
```

```
    }
}

class FabricaTemaEscuro implements FabricaUI {
    public Botao criarBotao() {
        return new BotaoEscuro();
    }

    public CaixaDialogo criarDialogo() {
        return new DialogoEscuro();
    }
}

class Tela {
    private final Botao botao;
    private final CaixaDialogo dialogo;

    Tela(FabricaUI fabrica) {
        botao = fabrica.criarBotao();
        dialogo = fabrica.criarDialogo();
    }

    void desenhar() {
        botao.renderizar();
        dialogo.abrir();
    }
}

public class Main {
    public static void main(String[] args) {
        FabricaUI fabrica = new FabricaTemaEscuro();
        Tela tela = new Tela(fabrica);
        tela.desenhar();
    }
}
```

Por que isso é uma família?

A fábrica escura produz tanto `BotaoEscuro` quanto `DialogoEscuro`. O cliente recebe ambos através das interfaces. Assim, a decisão sobre o tema fica concentrada na fábrica selecionada.

Vantagens

- Mantém os produtos de uma mesma família compatíveis.
- Permite trocar toda a família de produtos alterando a fábrica escolhida.
- Evita que o cliente dependa diretamente de classes concretas.

Limitações

- Adicionar uma nova família costuma ser simples: cria-se outra fábrica concreta.
- Adicionar um novo tipo de produto à interface da fábrica pode exigir alterações em todas as fábricas concretas.
- Pode ser excessivo quando não existe uma família real de objetos relacionados.

Quando usar

Use quando a aplicação possui conjuntos de produtos que devem variar juntos: temas de interface, componentes por sistema operacional, famílias de conexão ou conjuntos de objetos para diferentes ambientes.

> Diferença essencial
> Factory Method normalmente concentra a criação de um produto em um método redefinido. Abstract Factory oferece uma interface para criar vários produtos relacionados, formando uma família.

3. Builder

Problema que resolve

Construtores com muitos parâmetros ficam difíceis de ler e de usar corretamente. Isso piora quando vários parâmetros têm o mesmo tipo, quando muitos são opcionais ou quando a construção exige validações e etapas.

Ideia central

Builder separa a construção de um objeto da sua representação final. O cliente configura o objeto por etapas e, ao chamar `build()`, recebe a instância construída. Em Java, é comum implementar o padrão com uma classe interna estática chamada `Builder`.

Exemplo em Java

```
public class Usuario {
    private final String nome;
    private final String email;
    private final int idade;
    private final String telefone;
    private final boolean receberNotificacoes;

    private Usuario(Builder builder) {
        this.nome = builder.nome;
        this.email = builder.email;
        this.idade = builder.idade;
        this.telefone = builder.telefone;
        this.receberNotificacoes = builder.receberNotificacoes;
    }

    public static class Builder {
        private final String nome;
        private final String email;
        private int idade = 0;
        private String telefone = "";
        private boolean receberNotificacoes = true;

        public Builder(String nome, String email) {
            this.nome = nome;
            this.email = email;
        }

        public Builder idade(int idade) {
            if (idade < 0) {
                throw new IllegalArgumentException("Idade inválida");
            }
            this.idade = idade;
            return this;
        }

        public Builder telefone(String telefone) {
            this.telefone = telefone;
            return this;
        }

        public Builder receberNotificacoes(boolean valor) {
            this.receberNotificacoes = valor;
            return this;
        }

        public Usuario build() {
            if (nome == null || nome.isBlank()) {
                throw new IllegalStateException("Nome obrigatório");
            }
            if (email == null || !email.contains("@")) {
                throw new IllegalStateException("E-mail inválido");
            }
            return new Usuario(this);
        }
    }

    @Override
    public String toString() {
```

```
        return "Usuario{nome='" + nome + "', email='" + email
                + "', idade=" + idade + ", telefone='" + telefone
                + "', receberNotificacoes=" + receberNotificacoes + "}";
    }

    public static void main(String[] args) {
        Usuario usuario = new Usuario.Builder("Ana", "ana@email.com")
                .idade(23)
                .telefone("71999990000")
                .receberNotificacoes(false)
                .build();

        System.out.println(usuario);
    }
}
```

Entendendo a chamada encadeada

Cada método de configuração altera um campo do Builder e retorna `this`. Isso permite encadear chamadas. O método `build()` valida os dados e cria o objeto final.

```
Usuario usuario = new Usuario.Builder("Ana", "ana@email.com")
        .idade(23)
        .telefone("71999990000")
        .build();
```

Por que os campos são final?

No exemplo, os campos do objeto final são definidos no construtor privado e não possuem setters. Depois de construído, o objeto não permite alterar esses campos diretamente. Isso favorece a imutabilidade e reduz mudanças acidentais de estado.

Vantagens

- Torna a criação de objetos com muitos parâmetros mais legível.
- Distingue parâmetros obrigatórios (no construtor do Builder) dos opcionais (métodos de configuração).
- Centraliza validações antes de criar o objeto final.
- Pode facilitar a criação de objetos imutáveis.

Limitações

- Exige código adicional e pode ser desnecessário para objetos pequenos.
- É preciso decidir cuidadosamente quais campos são obrigatórios e quais têm valores padrão.
- A validação deve refletir as regras reais do domínio, não apenas verificações superficiais.

Quando usar

Use em objetos de configuração, requisições, relatórios, usuários, pedidos ou outros objetos com muitos campos opcionais. Não é obrigatório para toda classe com dois ou três atributos.

4. Prototype

Problema que resolve

Em alguns casos, criar um objeto do zero é custoso ou exige reconstruir uma configuração complexa. Se já existe um objeto configurado de forma adequada, pode ser conveniente criar uma cópia e modificar apenas alguns dados.

Ideia central

Prototype cria objetos por meio da cópia de uma instância existente. O padrão não significa apenas atribuir uma variável a outra: isso copia a referência, não cria um novo objeto. É necessário implementar uma operação de cópia.

Exemplo em Java usando um método de cópia

```
class ConfiguracaoRelatorio {
    private String titulo;
    private String formato;
    private boolean incluirGraficos;

    public ConfiguracaoRelatorio(String titulo, String formato,
                                 boolean incluirGraficos) {
        this.titulo = titulo;
        this.formato = formato;
        this.incluirGraficos = incluirGraficos;
    }

    // Cria um novo objeto com os mesmos valores.
    public ConfiguracaoRelatorio copiar() {
        return new ConfiguracaoRelatorio(
                this.titulo,
                this.formato,
                this.incluirGraficos
        );
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public void setFormato(String formato) {
        this.formato = formato;
    }

    @Override
    public String toString() {
        return "ConfiguracaoRelatorio{titulo='" + titulo
                + "', formato='" + formato
                + "', incluirGraficos=" + incluirGraficos + "}";
    }
}

public class Main {
    public static void main(String[] args) {
        ConfiguracaoRelatorio modelo =
                new ConfiguracaoRelatorio("Relatório mensal", "PDF", true);

        ConfiguracaoRelatorio copia = modelo.copiar();
        copia.setTitulo("Relatório trimestral");
        copia.setFormato("HTML");

        System.out.println(modelo);
        System.out.println(copia);
        System.out.println(modelo == copia); // false
    }
}
```

O que o exemplo demonstra?

O método `copiar()` retorna outra instância. Alterar o título da cópia não altera o título do modelo, porque os dois objetos são distintos. A comparação com == retorna `false`, pois as referências apontam

para objetos diferentes.

Cópia rasa e cópia profunda

Cópia rasa (shallow copy)

Cria um novo objeto, mas referências internas podem continuar apontando para os mesmos objetos. Se um campo é uma lista ou outro objeto mutável, modelo e cópia podem compartilhar esse conteúdo.

Cópia profunda (deep copy)

Copia também os objetos internos que precisam ser independentes. Pode exigir lógica adicional, especialmente quando há grafos de objetos, referências circulares ou recursos externos.

> Atenção em Java
> O método `Object.clone()` existe, mas tem limitações de design e normalmente produz cópia rasa por padrão. Uma operação de cópia explícita, um construtor de cópia ou uma biblioteca bem escolhida pode ser mais clara.

Vantagens

- Reaproveita configurações existentes.
- Pode reduzir o custo de criar objetos complexos repetidamente.
- Permite registrar protótipos e cloná-los conforme a necessidade.

Limitações

- É necessário definir claramente o que deve ser compartilhado e o que deve ser copiado.
- Cópia profunda pode ser complexa.
- Pode ser inadequado para objetos que mantêm conexões, arquivos abertos ou outros recursos externos.

Quando usar

Use quando existem modelos pré-configurados, objetos semelhantes são criados com frequência ou a construção do zero é custosa. Exemplos: configurações de relatórios, documentos-base, elementos de editor e objetos de simulação.

5. Singleton

Problema que resolve

Algumas classes representam um recurso que deve ter uma única instância dentro de uma aplicação, como um registro de configuração ou um coordenador compartilhado. Criar várias instâncias independentes pode causar inconsistência.

Ideia central

Singleton controla a criação da própria instância e fornece um ponto de acesso global. Em Java, uma forma simples e segura é usar um `enum`, especialmente quando não há necessidade de herança ou de inicialização especial.

Implementação recomendada para muitos casos: enum

```
public enum ConfiguracaoApp {
    INSTANCIA;

    private String idioma = "pt-BR";

    public String getIdioma() {
        return idioma;
    }

    public void setIdioma(String idioma) {
        this.idioma = idioma;
    }
}

public class Main {
    public static void main(String[] args) {
        ConfiguracaoApp a = ConfiguracaoApp.INSTANCIA;
        ConfiguracaoApp b = ConfiguracaoApp.INSTANCIA;

        a.setIdioma("pt-PT");

        System.out.println(b.getIdioma()); // pt-PT
        System.out.println(a == b);        // true
    }
}
```

Como funciona?

A enumeração possui uma constante `INSTANCIA`. O Java garante uma única instância de cada constante enum no processo. Assim, as duas variáveis referenciam o mesmo objeto.

Alternativa: inicialização antecipada

```
public final class Registro {
    private static final Registro INSTANCIA = new Registro();

    private Registro() {
        // Impede criação externa com new.
    }

    public static Registro getInstancia() {
        return INSTANCIA;
    }
}
```

A instância é criada quando a classe é inicializada. A inicialização de classes Java é sincronizada pela JVM, portanto essa abordagem é segura para threads no contexto habitual.

Alternativa: inicialização sob demanda com holder

```
public final class ServicoCompartilhado {
    private ServicoCompartilhado() {}

    private static class Holder {
```

```
        private static final ServicoCompartilhado INSTANCIA =
                new ServicoCompartilhado();
    }

    public static ServicoCompartilhado getInstancia() {
        return Holder.INSTANCIA;
    }
}
```

A classe interna só é inicializada quando `getInstancia()` é chamada pela primeira vez. Isso permite inicialização tardia sem escrever sincronização manual.

Singleton e concorrência

Uma instância única não torna automaticamente seus campos seguros para acesso concorrente. Se várias threads modificarem o estado compartilhado, ainda pode ser necessário usar imutabilidade, sincronização, locks ou classes concorrentes.

Vantagens

- Garante um único objeto acessível por um ponto conhecido.
- Pode centralizar acesso a estado ou recurso compartilhado.
- As implementações idiomáticas do Java podem ser seguras para threads na criação da instância.

Limitações e cuidados

- Cria dependência global, tornando o código mais difícil de testar e substituir.
- Pode esconder dependências que seriam mais claras como parâmetros ou serviços injetados.
- Não deve ser usado apenas para evitar passar objetos entre classes.
- Em aplicações com múltiplos processos, servidores ou classloaders, “única instância” não significa necessariamente uma única instância em todo o sistema.

Quando usar

Considere quando a unicidade for um requisito real e bem delimitado. Em aplicações modernas, prefira frequentemente injeção de dependências com escopo singleton gerenciado pelo framework, quando disponível, em vez de criar um Singleton global manual.

6. Comparação rápida

Padrão	Foco	Pergunta que ajuda a identificar	Exemplo
Factory Method	Criar um produto por meio de um método redefinível	Qual implementação deve ser criada por este criador?	Canal de notificação
Abstract Factory	Criar uma família de produtos relacionados	Quais objetos precisam variar juntos?	Tema claro/escuro
Builder	Construir um objeto em etapas	Como tornar legível a criação com muitos parâmetros?	Objeto de configuração
Prototype	Criar cópias de um objeto existente	Posso partir de um modelo já configurado?	Configuração de relatório
Singleton	Controlar a quantidade de instâncias	A unicidade é um requisito real?	Configuração compartilhada

Como distinguir Factory Method e Abstract Factory

Esta é uma das confusões mais comuns. Factory Method costuma delegar a escolha de um produto a uma implementação de método. Abstract Factory apresenta um conjunto de métodos para produzir diferentes tipos de produto de uma família. Uma Abstract Factory pode inclusive implementar seus métodos usando Factory Methods internamente.

Como distinguir Builder e Abstract Factory

Builder se preocupa com o processo de construção de um objeto, frequentemente com várias etapas e opções. Abstract Factory se preocupa com a seleção de uma família de objetos relacionados. Um Builder pode ser usado para construir um produto que, por sua vez, foi selecionado por uma fábrica.

Como distinguir Prototype e Builder

Prototype começa com um objeto existente e o copia. Builder começa com dados e configurações e monta uma nova instância. Se a cópia precisa ser independente, pense também na diferença entre cópia rasa e profunda.

Como distinguir Singleton e uma classe com métodos estáticos

Métodos estáticos não exigem necessariamente uma instância. Singleton fornece uma instância única, que pode manter estado e implementar interfaces. Porém, se não há estado nem necessidade de polimorfismo, uma classe utilitária com métodos estáticos pode ser mais simples.

7. Boas práticas ao aplicar os padrões

- Programe para interfaces: quando fizer sentido, faça o cliente depender de abstrações, não de classes concretas.
- Evite abstrações prematuras: introduza o padrão quando houver um problema real de criação, variação ou manutenção.
- Nomeie com intenção: nomes como `FabricaTemaEscuro` ou `criarNotificacao()` ajudam a comunicar o papel das classes.
- Separe responsabilidades: criação de objetos, regras de negócio e persistência não precisam estar concentradas na mesma classe.
- Teste o comportamento: padrões devem facilitar a evolução e os testes, não ser apenas uma demonstração de conhecimento.
- Considere o contexto: frameworks Java frequentemente oferecem injeção de dependências, fábricas, escopos e mecanismos de configuração.

8. Exercícios para praticar

Exercício 1 — Factory Method

Crie uma interface `Exportador` com o método `exportar(String conteudo)`. Implemente exportadores para PDF e CSV. Depois, crie um serviço base com um Factory Method e subclasses que escolham cada exportador.

Exercício 2 — Abstract Factory

Crie uma fábrica de componentes para duas plataformas fictícias. Cada fábrica deve criar um botão e um campo de texto compatíveis. O cliente deve funcionar apenas com as interfaces.

Exercício 3 — Builder

Crie uma classe `Pedido` com cliente e número de itens obrigatórios, além de endereço, observação e desconto opcionais. Use Builder e valide que o desconto não seja negativo.

Exercício 4 — Prototype

Crie uma classe `Projeto` que tenha nome, lista de tarefas e configurações. Implemente cópia e teste se modificar a lista da cópia afeta ou não o original. Explique se a implementação é rasa ou profunda.

Exercício 5 — Singleton

Implemente uma classe que represente configurações de aplicação. Compare a versão com construtor privado e método estático com a versão baseada em enum. Explique como cada uma se comporta quando chamada por duas variáveis.

Desafio integrador

Projete um pequeno sistema de relatórios: uma Abstract Factory seleciona a família de exportadores para diferentes ambientes; um Builder monta as opções do relatório; Prototype duplica uma configuração-base; e um serviço comum usa Factory Method para criar o exportador específico. Avalie se todos esses padrões são realmente necessários antes de implementar.

9. Checklist de revisão

- Consigo explicar o problema que cada padrão resolve sem decorar a definição?

- Consigo identificar produto, fábrica/criador e cliente em Factory Method?
- Consigo explicar por que Abstract Factory cria famílias de produtos compatíveis?
- Consigo justificar o uso de Builder em vez de um construtor comum?
- Sei por que atribuir uma variável a outra não cria uma cópia em Java?
- Sei diferenciar cópia rasa de cópia profunda?
- Sei explicar como Singleton controla a instância e por que estado compartilhado ainda exige cuidado com concorrência?
- Consigo citar pelo menos uma limitação de cada padrão?

Resumo final

Os cinco padrões tratam de decisões diferentes. Factory Method delega a criação de um produto; Abstract Factory cria famílias relacionadas; Builder organiza a construção por etapas; Prototype parte de uma instância existente para criar cópias; Singleton controla a unicidade de uma instância. O melhor padrão é aquele que resolve um problema concreto com o menor nível de complexidade necessário.

Sugestão de estudo: leia a intenção de cada padrão, execute os exemplos, altere uma parte do código e observe o impacto. Depois, faça os exercícios sem consultar o material e volte ao guia para revisar as diferenças.